기계학습

Support Vector Machine (SVM)

Ji-Hyeong Han

(jhhan@seoultech.ac.kr)

Dept. of Computer Science and Engineering

SEOULTECH

Contents

1. Linear SVM Classification
2. Nonlinear SVM Classification
3. SVM Regression
4. Under the Hood

Introduction

- A *Support Vector Machine* (SVM) is a powerful and versatile Machine Learning model, capable of performing linear or nonlinear classification, regression, and even outlier detection.

- SVMs are particularly well suited for classification of complex small- or medium-sized datasets.

Linear SVM Classification

Linear SVM Classification

- Fundamental idea behind SVMs

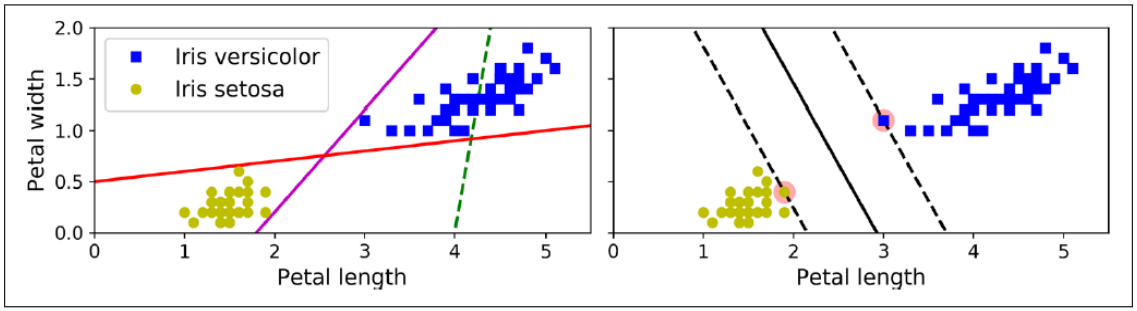


Figure 5-1. Large margin classification

- Two classes can clearly be separated easily with a straight line.
- There can be many decision boundaries.
  - Left plot: the two models (solid line decision boundaries) work perfectly on this training set, but their decision boundaries come so close to the instances that these models will probably not perform as well on new instances.
  - Right plot: the solid line represents the decision boundary of an SVM classifier; this line not only separates the two classes but also stays as far away from the closest training instances as possible.
- You can think of an SVM classifier as fitting the widest possible street (represented by the parallel dashed lines) between the classes.
- This is called *large margin classification*.

Linear SVM Classification

- Notice that adding more training instances “off the street” will not affect the decision boundary at all
  - It is fully determined (or “supported”) by the instances located on the edge of the street.
  - These instances are called the *support vectors.* They are circled in Figure 5-1.

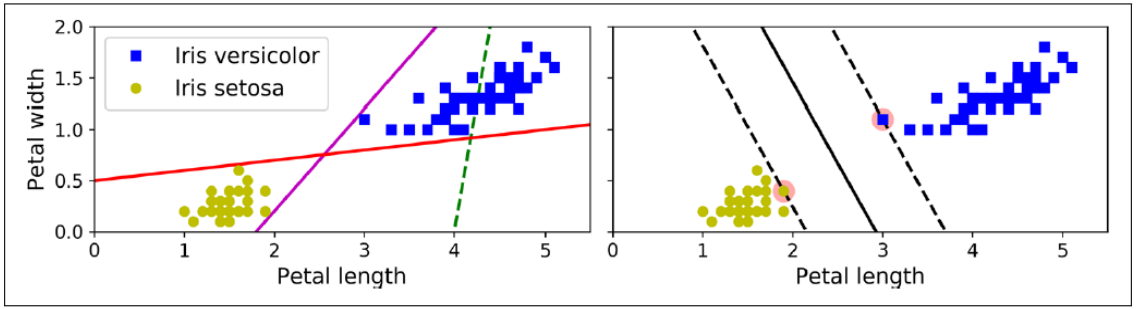


Figure 5-1. Large margin classification

Linear SVM Classification

- SVMs are sensitive to the feature scales.
  - Left plot: the vertical scale is much larger than the horizontal scale, so the widest possible street is close to horizontal.
  - Right plot: After feature scaling, the decision boundary in the right plot looks much better.

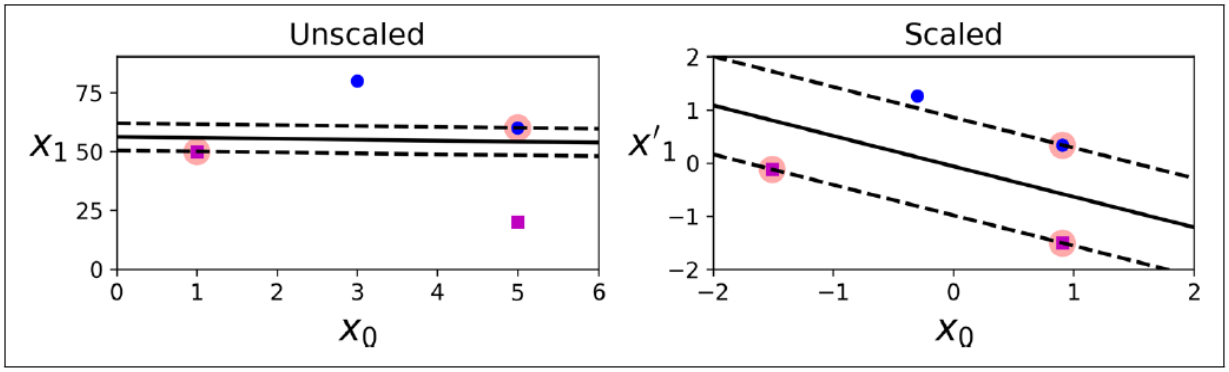


Figure 5-2. Sensitivity to feature scales

Soft Margin Classification

- Hard margin classification
  - If we strictly impose that all instances must be off the street and on the right side.

- Two main issues with hard margin classification:
  - First, it only works if the data is linearly separable.
  - Second, it is sensitive to outliers.

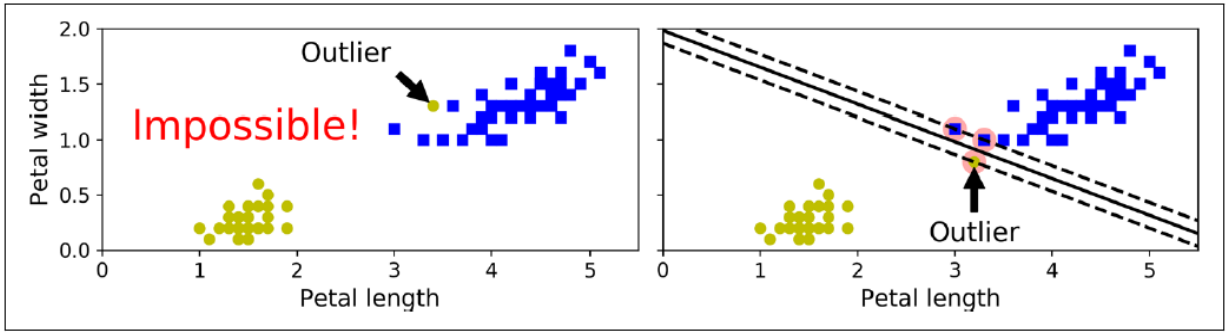


Figure 5-3. Hard margin sensitivity to outliers

Soft Margin Classification

- Soft margin classification
  - To avoid these issues, use a more flexible model.
  - The objective is to find a good balance between keeping the street as large as possible and limiting the *margin violations* (i.e., instances that end up in the middle of the street or even on the wrong side).

- Hyperparameter C of SVM model in Scikit-Learn
  - If we set it to a low value, then we end up with the model on the left of Figure 5-4.
  - With a high value, we get the model on the right.

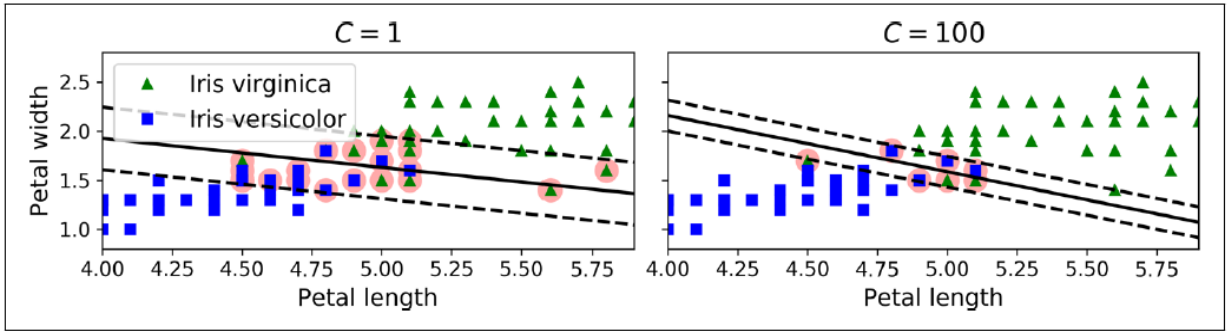


Figure 5-4. Large margin (left) versus fewer margin violations (right)

Soft Margin Classification

- Margin violations are bad.
- It's usually better to have few of them.
- However, in this case the model on the left has a lot of margin violations but will probably generalize better.
- If your SVM model is overfitting, you can try regularizing it by reducing C.

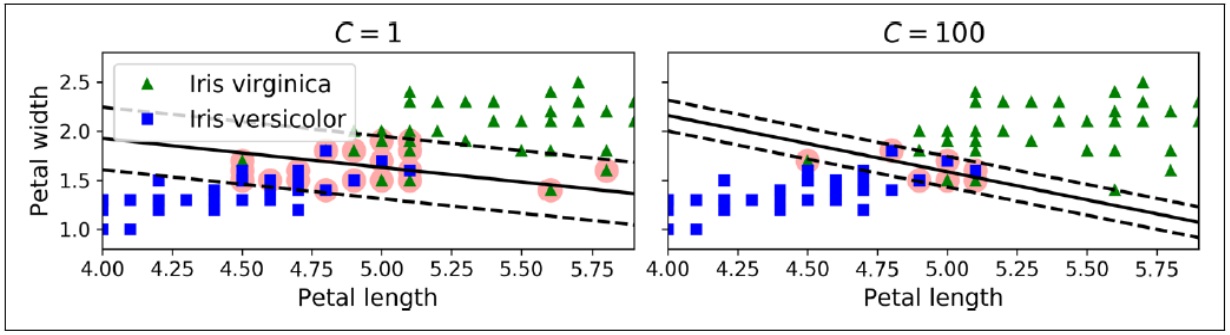


Figure 5-4. Large margin (left) versus fewer margin violations (right)

Soft Margin Classification

- The following Scikit-Learn code loads the iris dataset, scales the features, and then trains a linear SVM model (using the LinearSVC class with C=1 and the *hinge loss* function, described shortly) to detect *Iris virginica* flowers:

```
import numpy as np
from sklearn import datasets
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC

iris = datasets.load_iris()
X = iris["data"][:, (2, 3)]  # petal length, petal width
y = (iris["target"] == 2).astype(np.float64)  # Iris virginica

svm_clf = Pipeline([
        ("scaler", StandardScaler()),
        ("linear_svc", LinearSVC(C=1, loss="hinge")),
    ])

svm_clf.fit(X, y)

>>> svm_clf.predict([[5.5, 1.7]])
array([1.])
```

  - Unlike Logistic Regression classifiers, SVM classifiers do not output probabilities for each class.

Soft Margin Classification

- The LinearSVC class regularizes the bias term, so you should center the training set first by subtracting its mean.
  - This is automatic if you scale the data using the StandardScaler.

- Also make sure you set the loss hyperparameter to "hinge", as it is not the default value.

- Finally, for better performance, you should set the dual hyperparameter to False, unless there are more features than training instances (we will discuss duality later in the chapter).

Nonlinear SVM Classification

Nonlinear SVM Classification

- One approach to handling nonlinear datasets is to add more features, such as polynomial features; in some cases this can result in a linearly separable dataset.
  - Left plot: not linearly separable
  - Right plot: add $x_2 = (x_1)^2$, the resulting dataset is perfectly linearly separable

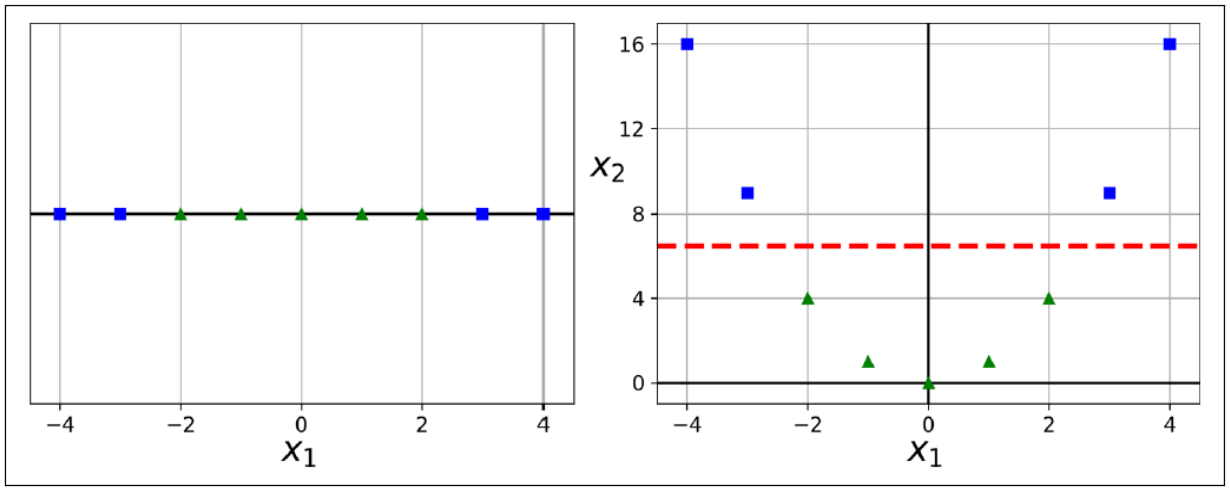


Figure 5-5. Adding features to make a dataset linearly separable

Nonlinear SVM Classification

- To implement this idea using Scikit-Learn, create a Pipeline containing a PolynomialFeatures transformer.
  - Let's test this on the moons dataset:

```
from sklearn.datasets import make_moons
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures

X, y = make_moons(n_samples=100, noise=0.15)
polynomial_svm_clf = Pipeline([
        ("poly_features", PolynomialFeatures(degree=3)),
        ("scaler", StandardScaler()),
        ("svm_clf", LinearSVC(C=10, loss="hinge"))
    ])

polynomial_svm_clf.fit(X, y)
```

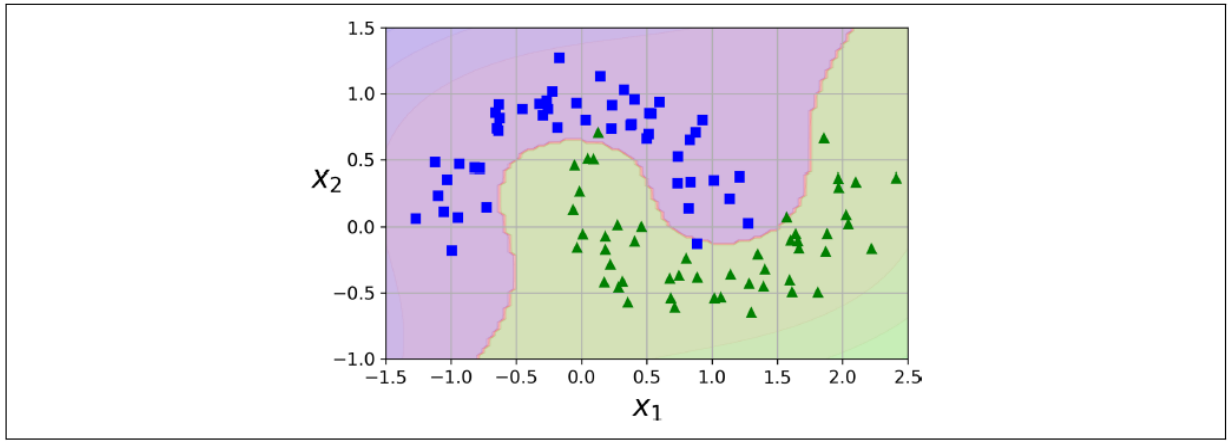


Figure 5-6. Linear SVM classifier using polynomial features

Polynomial Kernel

- Two issues of adding polynomial features:
  - First, a low polynomial degree, this method cannot deal with very complex datasets.
  - Second, with a high polynomial degree it creates a huge number of features, making the model too slow.

- Fortunately, when using SVMs you can apply an almost miraculous mathematical technique called the *kernel trick.*

- The kernel trick makes it possible to get the same result as if you had added many polynomial features, even with very high-degree polynomials, without actually having to add them.
  - So there is no combinatorial explosion of the number of features because you don’t actually add any features.

Polynomial Kernel

- Let's test it on the moons dataset:

```
from sklearn.svm import SVC
poly_kernel_svm_clf = Pipeline([
        ("scaler", StandardScaler()),
        ("svm_clf", SVC(kernel="poly", degree=3, coef0=1, C=5))
    ])
poly_kernel_svm_clf.fit(X, y)
```

- This code trains an SVM classifier using a third-degree polynomial kernel
- If your model is overfitting, reduce the polynomial degree.
- If your model is underfitting, increase the polynomial degree.
- Hyperparameter coef0 controls how much the model is influenced by high-degree polynomials vs. low-degree polynomials.

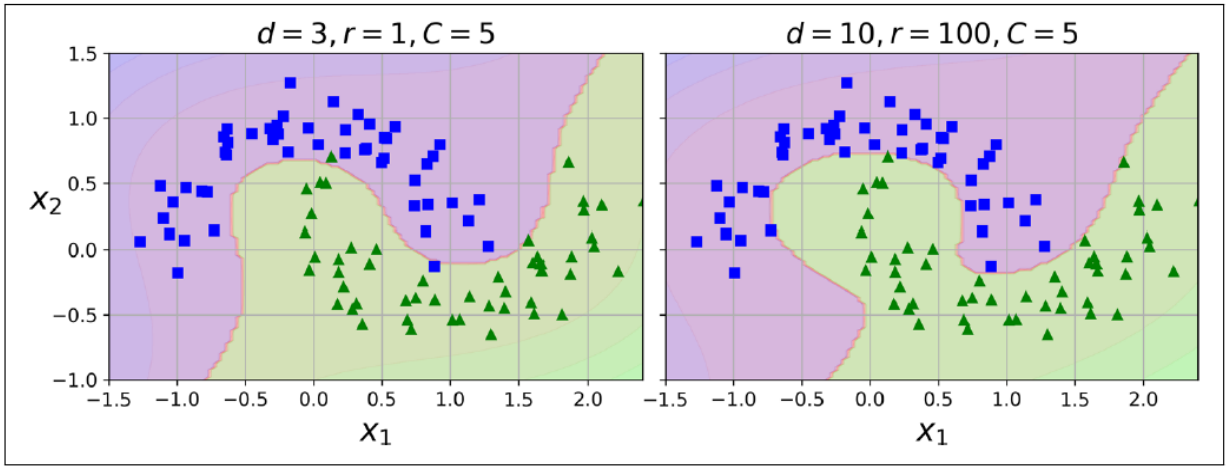


Figure 5-7. SVM classifiers with a polynomial kernel

Similarity Features

- Another technique to tackle nonlinear problems is to add features computed using a *similarity function*, which measures how much each instance resembles a particular *landmark*.
  - Ex: let's take the 1D dataset discussed earlier and add two landmarks to it at $x_1 = -2$ and $x_1 = 1$.
  - Next, let's define the similarity function to be the Gaussian *Radial Basis Function* (RBF) with $\gamma = 0.3$. This is a bell-shaped function varying from 0 (very far away from the landmark) to 1 (at the landmark).

Equation 5-1. Gaussian RBF

$$\phi_\gamma(\mathbf{x}, \ell) = \exp\left(-\gamma \| \mathbf{x} - \ell \|^2\right)$$

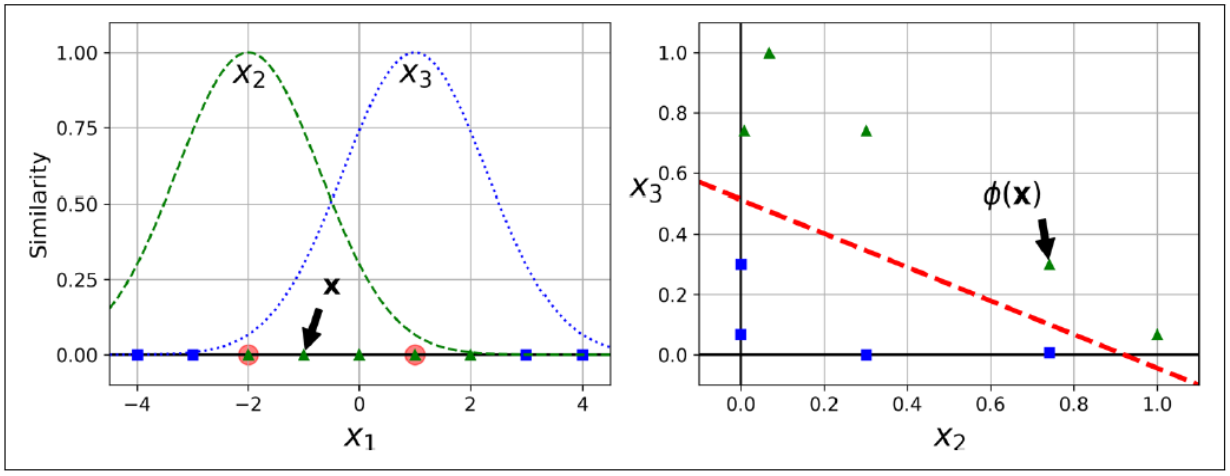


Figure 5-8. Similarity features using the Gaussian RBF

Similarity Features

- Now we are ready to compute the new features.
- let's look at the instance $x_1 = -1$: it is located at a distance of 1 from the first landmark and 2 from the second landmark.
  - Therefore its new features are $x_2 = \exp\left(-0.3 \times 1^2\right) \approx 0.74$ and $x_3 = \exp\left(-0.3 \times 2^2\right) \approx 0.30$

- Right plot: the transformed dataset (dropping the original features). As you can see, it is now linearly separable.

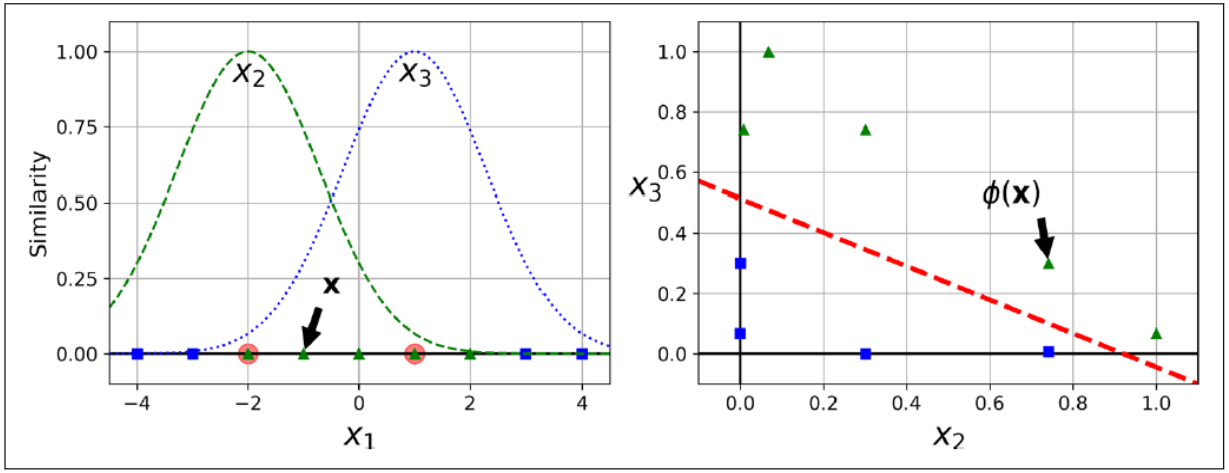


Figure 5-8. Similarity features using the Gaussian RBF

Similarity Features

- How to select the landmarks?
  - The simplest approach is to create a landmark at the location of each and every instance in the dataset.
  - Doing that creates many dimensions and thus increases the chances that the transformed training set will be linearly separable.
  - The downside is that a training set with *m* instances and *n* features gets transformed into a training set with *m* instances and *m* features (assuming you drop the original features).
  - If your training set is very large, you end up with an equally large number of features.

Gaussian RBF Kernel

- Just like the polynomial features method, the similarity features method can be useful with any Machine Learning algorithm, but it may be computationally expensive to compute all the additional features, especially on large training sets.

- Once again the kernel trick does its SVM magic, making it possible to obtain a similar result as if you had added many similarity features.

- Let's try the SVC class with the Gaussian RBF kernel:

```
rbf_kernel_svm_clf = Pipeline([
        ("scaler", StandardScaler()),
        ("svm_clf", SVC(kernel="rbf", gamma=5, C=0.001))
    ])
rbf_kernel_svm_clf.fit(X, y)
```

Gaussian RBF Kernel

- Hyperparameter γ
  - Increasing γ makes bell-shaped curve narrower, so each instance's range of influence is smaller: the decision boundary ends up being more irregular, wiggling around individual instances. → possibly overfitting
  - Decreasing γ makes bell-shaped curve wider, so each instance has a larger range of influence: the decision boundary ends up smoother. → possibly underfitting

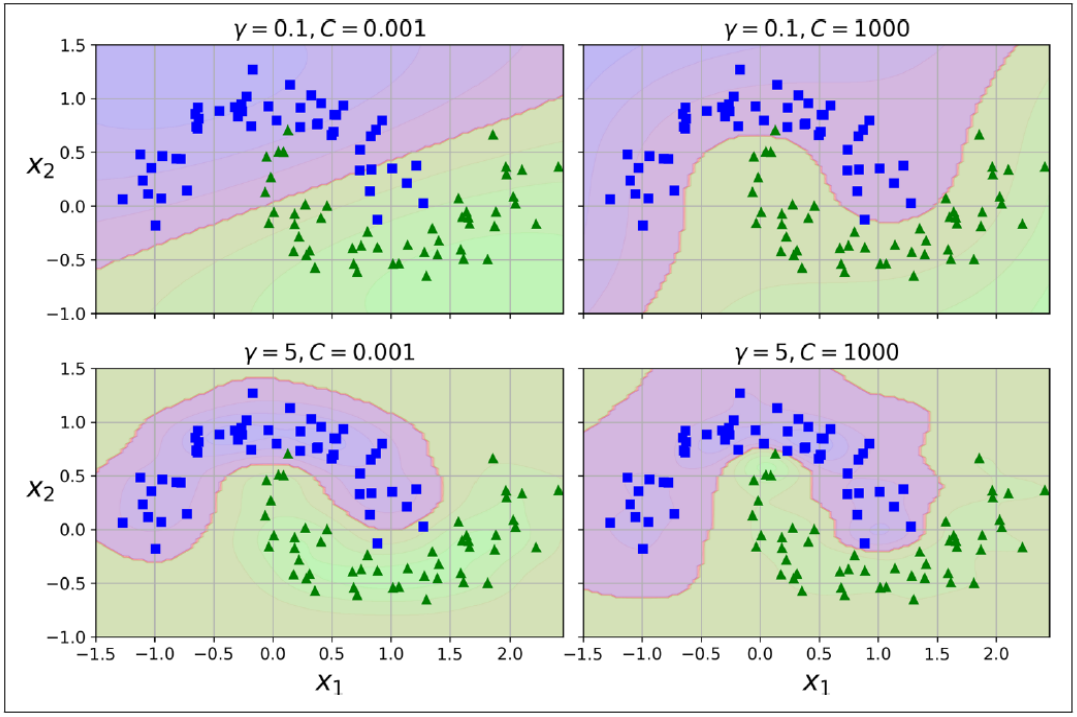


Figure 5-9. SVM classifiers using an RBF kernel

Other Kernels

- Other kernels exist but are used much more rarely.

- Some kernels are specialized for specific data structures.
  - *String kernels* are sometimes used when classifying text documents or DNA sequences (e.g., using the *string subsequence kernel* or kernels based on the *Levenshtein distance*).

- How to choose the kernel?
  - As a rule of thumb, you should always try the linear kernel first, especially if the training set is very large or if it has plenty of features.
  - If the training set is not too large, you should also try the Gaussian RBF kernel; it works well in most cases.
  - Then if you have spare time and computing power, you can experiment with a few other kernels, using cross-validation and grid search.

Computational Complexity

- The LinearSVC class is based on the liblinear library.
  - It does not support the kernel trick, but it scales almost linearly with the number of training instances and the number of features.
  - $O(m \times n)$
- The SVC class is based on the libsvm library, which implements an algorithm that supports the kernel trick.
  - The training time complexity is between $O(m^2 \times n)$ and $O(m^3 \times n)$
  - This means that it gets dreadfully slow when the number of training instances gets large.
  - This algorithm is perfect for complex small or medium-sized training sets.

Table 5-1. Comparison of Scikit-Learn classes for SVM classification

Class	Time complexity	Out-of-core support	Scaling required	Kernel trick
`LinearSVC`	$O(m \times n)$	No	Yes	No
`SGDClassifier`	$O(m \times n)$	Yes	Yes	No
`SVC`	$O(m^2 \times n)$ to $O(m^3 \times n)$	No	Yes	Yes

SVM Regression

SVM Regression

- SVM supports linear and nonlinear regression.

- To use SVMs for regression instead of classification, the trick is to reverse the objective:
  - Instead of trying to fit the largest possible street between two classes while limiting margin violations, SVM Regression tries to fit as many instances as possible *on* the street while limiting margin violations (i.e., instances *off* the street).
  - The width of the street is controlled by a hyperparameter, ϵ.

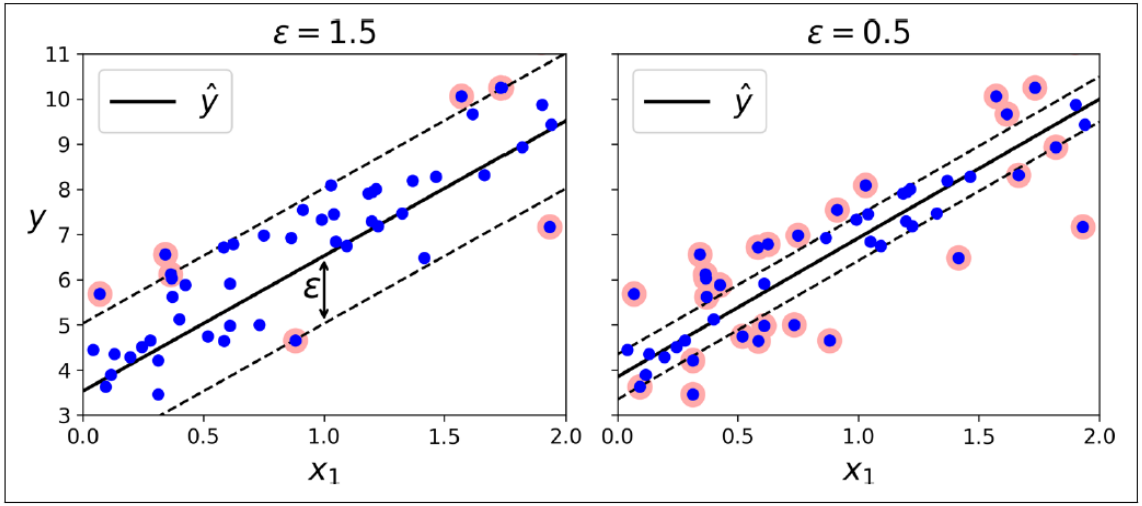


Figure 5-10. SVM Regression

SVM Regression

- Adding more training instances within the margin does not affect the model's predictions; thus, the model is said to be *ϵ-insensitive.*

- You can use Scikit-Learn's LinearSVR class to perform linear SVM Regression:

```
from sklearn.svm import LinearSVR

svm_reg = LinearSVR(epsilon=1.5)
svm_reg.fit(X, y)
```

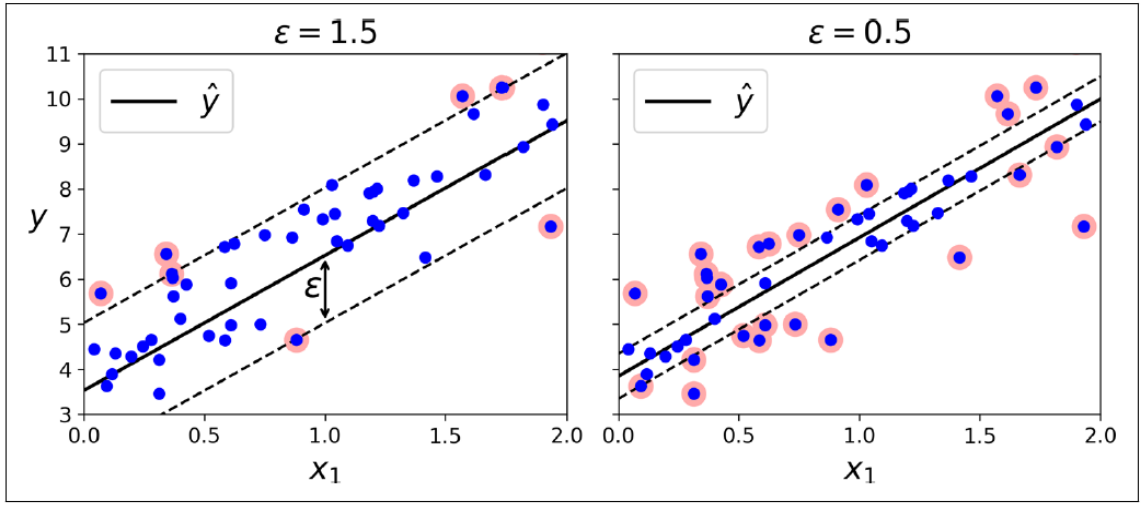


Figure 5-10. SVM Regression

SVM Regression

- To tackle nonlinear regression tasks, you can use a kernelized SVM model.

```
from sklearn.svm import SVR

svm_poly_reg = SVR(kernel="poly", degree=2, C=100, epsilon=0.1)
svm_poly_reg.fit(X, y)
```

[less regularization] [more regularization]

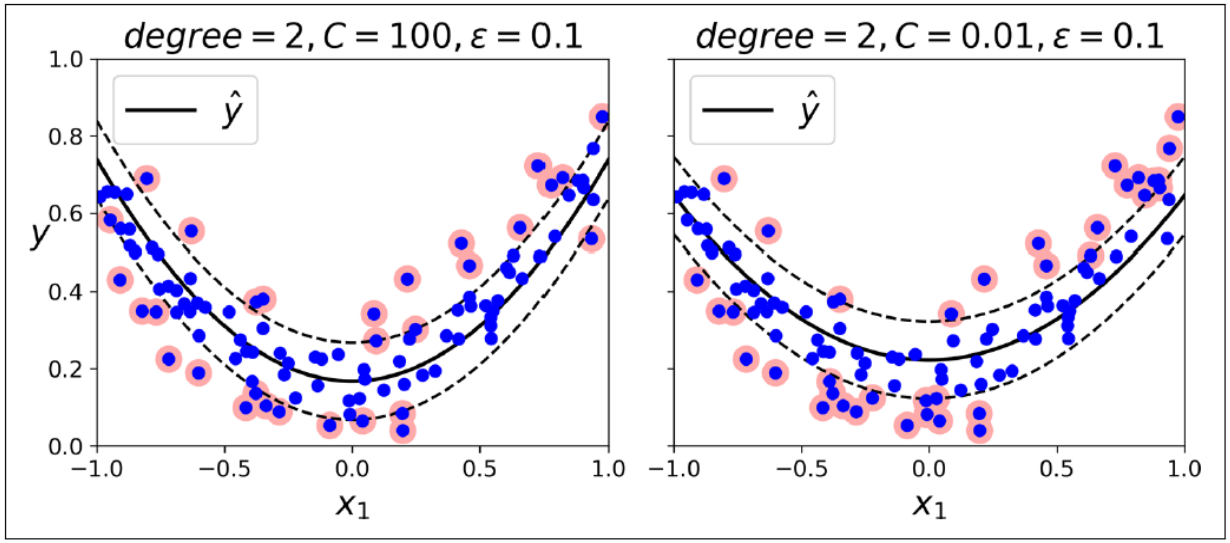


Figure 5-11. SVM Regression using a second-degree polynomial kernel

Under the Hood

Decision Function and Predictions

- Decision function of linear SVM classifier

$$\mathbf{w}^\intercal \mathbf{x} + b = w_1 x_1 + \cdots + w_n x_n + b$$

  - if the result is positive, the predicted class $\hat{y}$ is the positive class (1)
  - otherwise it is the negative class (0)

Equation 5-2. Linear SVM classifier prediction

$$\hat{y} = \begin{cases} 0 \text{ if } \mathbf{w}^\intercal \mathbf{x} + b < 0, \\ 1 \text{ if } \mathbf{w}^\intercal \mathbf{x} + b \geq 0 \end{cases}$$

- The decision boundary (solid line) is the set of points where the decision function is equal to 0

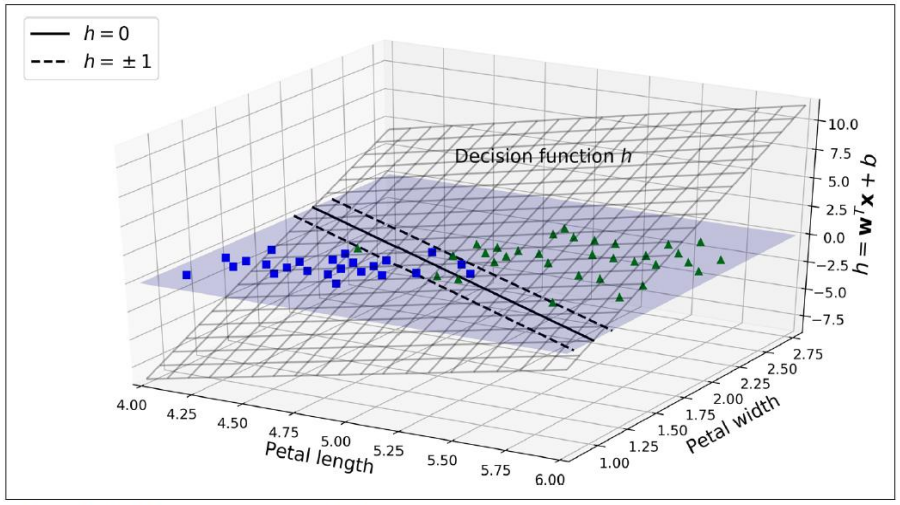


Figure 5-12. Decision function for the iris dataset

Decision Function and Predictions

- The dashed lines represent the points where the decision function is equal to 1 or -1
  - They are parallel and at equal distance to the decision boundary
  - They form a margin around the decision boundary

- Training a linear SVM classifier means finding the values of $\mathbf{w}$ and b that make this margin as wide as possible while avoiding margin violations (hard margin) or limiting them (soft margin).

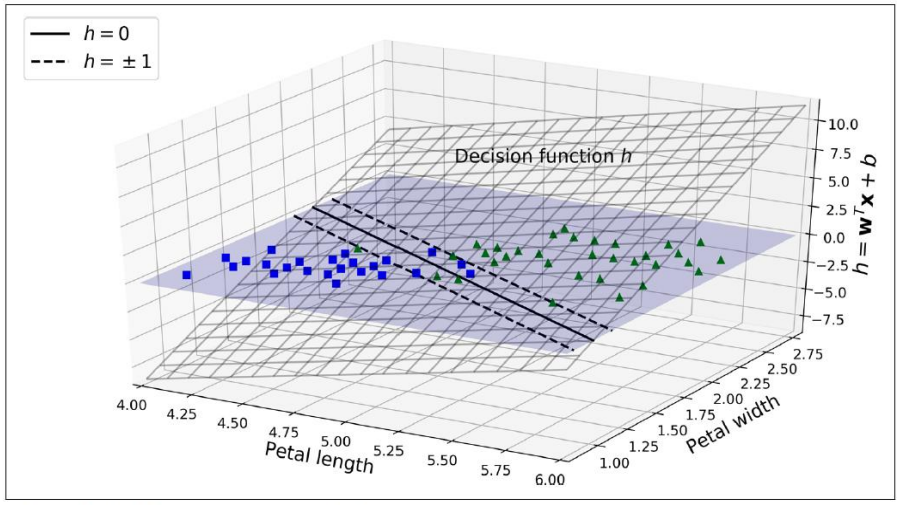


Figure 5-12. Decision function for the iris dataset

Training Objective

- The smaller the weight vector $\mathbf{w}$, the larger the margin

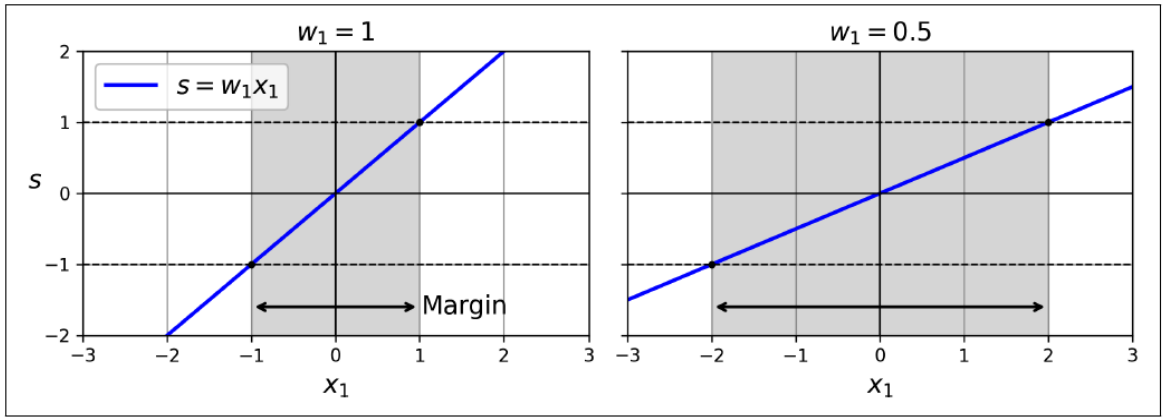


Figure 5-12. A smaller weight vector results in a larger margin

- So we want to minimize $||\mathbf{w}||$ to get a large margin.
- If we want to avoid any margin violations (hard margin), then we need the decision function to be greater than 1 for all positive training instances and lower than -1 for negative training instances.
  - So we define $t^{(i)} = -1$ for negative instances (i.e. $y^{(i)} = 0$) and $t^{(i)} = 1$ for positive instances (i.e. $y^{(i)} = 1$), then we can express this constraint as follows:
    $t^{i}\left(\mathbf{w}^{\mathrm{T}}\mathbf{x}^{(i)} + b\right) \geq 1$ for all instances.

Training Objective

- The hard margin linear SVM classifier objective as the constrained optimization problem

Equation 5-3. Hard margin linear SVM classifier objective

$$\underset{\mathbf{w}, b}{\text{minimize}} \quad \frac{1}{2}\mathbf{w}^{\mathsf{T}}\mathbf{w}$$

$$\text{subject to} \quad t^{(i)}\left(\mathbf{w}^{\mathsf{T}}\mathbf{x}^{(i)} + b\right) \geq 1 \quad \text{for } i = 1, 2, \cdots, m$$

  - Notice that we minimize $\frac{1}{2}\mathbf{w}^{\mathbf{T}}\mathbf{w}$, which is equal to $\frac{1}{2}||\mathbf{w}||^{\mathbf{2}}$, rather than minimize $||\mathbf{w}||$, because $\frac{1}{2}||\mathbf{w}||^{\mathbf{2}}$ has a nice, simple derivative, which is just $\mathbf{w}$, while $||\mathbf{w}||$ is not differentiable at $\mathbf{w} = 0$. Optimization algorithms work much better on differentiable functions.

Training Objective

- To get the soft margin objective, we need to introduce a slack variable $\zeta^{(i)} \geq 0$ for each instance
  - $\zeta^{(i)}$ measures how much i^{th} instance is allowed to violate the margin.

- We now have two conflicting objectives:
  - make the slack variables as small as possible to reduce the margin violations.
  - make $\frac{1}{2}\mathbf{w}^{\mathbf{T}}\mathbf{w}$ as small as possible to increase the margin.
  - This is where C hyperparameter comes in: it allows us to define the trade-off between these two objectives

Equation 5-4. Soft margin linear SVM classifier objective

$$\underset{\mathbf{w}, b, \zeta}{\text{minimize}} \quad \frac{1}{2}\mathbf{w}^{\mathsf{T}}\mathbf{w} + C\sum_{i=1}^{m}\zeta^{(i)}$$

$$\text{subject to} \quad t^{(i)}\left(\mathbf{w}^{\mathsf{T}}\mathbf{x}^{(i)} + b\right) \geq 1 - \zeta^{(i)} \quad \text{and} \quad \zeta^{(i)} \geq 0 \quad \text{for } i = 1, 2, \cdots, m$$

Quadratic Programming

- The hard margin and soft margin problems are both convex quadratic optimization problems with linear constraints.
  - Such problems are known as *Quadratic Programming* (QP) problems.

Equation 5-5. Quadratic Programming problem

$$\underset{\mathbf{p}}{\text{Minimize}} \quad \frac{1}{2}\mathbf{p}^\intercal \mathbf{H}\mathbf{p} + \mathbf{f}^\intercal \mathbf{p}$$

$$\text{subject to} \quad \mathbf{A}\mathbf{p} \le \mathbf{b} \qquad \mathbf{A}\,\mathbf{p} \le \mathbf{b} \text{ defines } n_c \text{ constraints: } \mathbf{p}^\intercal\,\mathbf{a}^{(i)} \le b^{(i)} \text{ for } i = 1, 2, \cdots, n_c,$$

where:

- $\mathbf{p}$ is an n_p-dimensional vector (n_p = number of parameters),
- $\mathbf{H}$ is an $n_p \times n_p$ matrix,
- $\mathbf{f}$ is an n_p-dimensional vector,
- $\mathbf{A}$ is an $n_c \times n_p$ matrix (n_c = number of constraints),
- $\mathbf{b}$ is an n_c-dimensional vector.

Equation 5-3. Hard margin linear SVM classifier objective

$$\underset{\mathbf{w}, b}{\text{minimize}} \quad \frac{1}{2}\mathbf{w}^\intercal\mathbf{w}$$

$$\text{subject to} \quad t^{(i)}\left(\mathbf{w}^\intercal\mathbf{x}^{(i)} + b\right) \ge 1 \quad \text{for } i = 1, 2, \cdots, m$$

- $n_p = n + 1$, where n is the number of features (the +1 is for the bias term).
- $n_c = m$, where m is the number of training instances.
- $\mathbf{H}$ is the $n_p \times n_p$ identity matrix, except with a zero in the top-left cell (to ignore the bias term).
- $\mathbf{f} = 0$, an n_p-dimensional vector full of 0s.
- $\mathbf{b} = -1$, an n_c-dimensional vector full of –1s.
- $\mathbf{a}^{(i)} = -t^{(i)}\,\dot{\mathbf{x}}^{(i)}$, where $\dot{\mathbf{x}}^{(i)}$ is equal to $\mathbf{x}^{(i)}$ with an extra bias feature $\dot{\mathbf{x}}_0 = 1$.

The Dual Problem

- Given a constrained optimization problem, known as the *primal problem*, it is possible to express a different but closely related problem, called its *dual problem.*
  - Under some conditions, the dual problem can have the same solution as the primal problem
  - Condition: the objective function is convex, and the inequality constraints are continuously differentiable and convex functions

- Luckily, the SVM problem happens to meet these conditions, so you can choose to solve the primal problem or the dual problem; both will have the same solution.

The Dual Problem

- Dual form the linear SVM objective

Equation 5-6. Dual form of the linear SVM objective

$$\underset{\alpha}{\text{minimize}} \frac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} \alpha^{(i)} \alpha^{(j)} t^{(i)} t^{(j)} \mathbf{x}^{(i)\intercal} \mathbf{x}^{(j)} - \sum_{i=1}^{m} \alpha^{(i)}$$

$$\text{subject to} \quad \alpha^{(i)} \geq 0 \quad \text{for } i = 1, 2, \cdots, m$$

  - Once you find the vector $\hat{\boldsymbol{\alpha}}$ that minimizes this equation, use the following equation to compute $\hat{\mathbf{w}}$ and $\hat{b}$ that minimizes the primal problem

Equation 5-7. From the dual solution to the primal solution

$$\hat{\mathbf{w}} = \sum_{i=1}^{m} \hat{\alpha}^{(i)} t^{(i)} \mathbf{x}^{(i)}$$

$$\hat{b} = \frac{1}{n_s} \sum_{\substack{i=1 \\ \hat{\alpha}^{(i)} > 0}}^{m} \left(t^{(i)} - \hat{\mathbf{w}}^{\intercal} \mathbf{x}^{(i)} \right)$$

- The dual problem makes the kernel trick possible, while the primal does not.

Kernelized SVMs

- Suppose you want to apply a second-degree polynomial transformation to a two dimensional training set (such as the moons training set), then train a linear SVM classifier on the transformed training set.

Equation 5-8. Second-degree polynomial mapping

$$\phi(\mathbf{x}) = \phi\left(\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}\right) = \begin{pmatrix} {x_1}^2 \\ \sqrt{2}\, x_1 x_2 \\ {x_2}^2 \end{pmatrix}$$

  - Notice that the transformed vector is 3D instead of 2D.

- Now let's look at what happens to a couple of 2D vectors, **a** and **b**, if we apply this second-degree polynomial mapping and then compute the dot product of the transformed vectors.

Equation 5-9. Kernel trick for a second-degree polynomial mapping

$$\begin{aligned} \phi(\mathbf{a})^\intercal \phi(\mathbf{b}) &= \begin{pmatrix} {a_1}^2 \\ \sqrt{2}\, a_1 a_2 \\ {a_2}^2 \end{pmatrix}^\intercal \begin{pmatrix} {b_1}^2 \\ \sqrt{2}\, b_1 b_2 \\ {b_2}^2 \end{pmatrix} = {a_1}^2 {b_1}^2 + 2 a_1 b_1 a_2 b_2 + {a_2}^2 {b_2}^2 \\ &= (a_1 b_1 + a_2 b_2)^2 = \left(\begin{pmatrix} a_1 \\ a_2 \end{pmatrix}^\intercal \begin{pmatrix} b_1 \\ b_2 \end{pmatrix}\right)^2 = (\mathbf{a}^\intercal \mathbf{b})^2 \end{aligned}$$

Kernelized SVMs

- The dot product of the transformed vectors is equal to the square of the dot product of the original vectors: $\phi(\mathbf{a})^{\mathrm{T}}\phi(\mathbf{b}) = \left(\mathbf{a}^{\mathrm{T}}\mathbf{b}\right)^2$
  - Here is the key insight of kernel trick of SVM

- If you apply the transformation ϕ to all training instances, then the dual problem will contain the dot product $\phi\left(\mathbf{x}^{(i)}\right)^{\mathrm{T}}\phi\left(\mathbf{x}^{(j)}\right)$.
  - But if ϕ is the second-degree polynomial transformation, then you can replace this dot product of transformed vectors simply by $\left(\mathbf{x}^{(i)\mathrm{T}}\mathbf{x}^{(j)}\right)^2$.
  - So you don't need to transform the training instances at all; just replace the dot product by its square

Equation 5-6. Dual form of the linear SVM objective

$$\underset{\alpha}{\text{minimize}}\ \frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{m}\alpha^{(i)}\alpha^{(j)}t^{(i)}t^{(j)}\mathbf{x}^{(i)\mathrm{T}}\mathbf{x}^{(j)} - \sum_{i=1}^{m}\alpha^{(i)}$$

$$\text{subject to}\quad \alpha^{(i)} \geq 0 \quad \text{for } i = 1, 2, \cdots, m$$

Kernelized SVMs

- The function $K(\mathbf{a}, \mathbf{b}) = \left(\mathbf{a}^{\mathrm{T}}\mathbf{b}\right)^2$ is a second-degree polynomial kernel.

- In machine learning, a *kernel* is a function capable of computing the dot product $\phi(\mathbf{a})^{\mathrm{T}}\phi(\mathbf{b})$, based only on the original vectors $\mathbf{a}$ and $\mathbf{b}$, without having to compute the transformation ϕ.

Equation 5-10. Common kernels

Linear:	$K(\mathbf{a}, \mathbf{b}) = \mathbf{a}^{\mathsf{T}}\mathbf{b}$
Polynomial:	$K(\mathbf{a}, \mathbf{b}) = (\gamma\mathbf{a}^{\mathsf{T}}\mathbf{b} + r)^d$
Gaussian RBF:	$K(\mathbf{a}, \mathbf{b}) = \exp\left(-\gamma\|\mathbf{a} - \mathbf{b}\|^2\right)$
Sigmoid:	$K(\mathbf{a}, \mathbf{b}) = \tanh(\gamma\mathbf{a}^{\mathsf{T}}\mathbf{b} + r)$

Online SVMs

- For linear SVM classifiers, one method for implementing an online SVM classifier is to use Gradient Descent to minimize the following cost function which is derived from the primal problem.

Equation 5-13. Linear SVM classifier cost function

$$J(\mathbf{w}, b) = \frac{1}{2}\mathbf{w}^\intercal\mathbf{w} \quad + \quad C\sum_{i=1}^{m} max\left(0, 1 - t^{(i)}\left(\mathbf{w}^\intercal\mathbf{x}^{(i)} + b\right)\right)$$

  - Unfortunately, Gradient Descent converges much more slowly than the methods based on QP.
  - The first sum in the cost function will push the model to have a small weight vector **w**, leading to a larger margin.
  - The second sum computes the total of all margin violations. An instance's margin violation is equal to 0 if it is located off the street and on the correct side, or else it is proportional to the distance to the correct side of the street.
  - Minimizing the second term ensures that the model makes the margin violations as small and as few as possible.

Online SVMs

- Hinge loss
  - The function *max*(0, 1 – *st*) is called the *hinge loss* function

